

# CMS OSPO Outbound Review Checklist

---

## Tier 1: One-Time Release

---

This is a review process to approve CMS-developed software to be released open source. If you would like your repository to be released, please complete the following steps.

### Prerequisites

[State the Benefits of Open Sourcing the Project](#)

[State the Risks of Open Sourcing the Project](#)

[Questions](#)

[Code Review](#)

[Code Analysis](#)

[Review Licensing](#)

[Review Commit History](#)

[Review Repository Hygiene](#)

[Review Project Metadata](#)

[Review Repository Details](#)

[Additional Notes & Questions](#)

[Sign off on risk acceptance of open-sourcing the software product](#)

[Flipping the Switch: Making the Repository Public](#)

### Prerequisites

**Your project should be stored in a GitHub repository** If you are new to GitHub or repositories, checkout the OSPO's [Beginner Guide to Creating GitHub Repos](#) before moving on to the next steps.

Does your repository align with the requirements of a Tier 1 project? To verify:

- Review the flowchart or use `tier-determiner.py` provided in the [README.md](#).
- Read more about [Tier 1](#) and the overall [CMS OSPO maturity model framework](#).

### Results

*Insert Review Here*

### State the Benefits of Open Sourcing the Project

- ☐ **Cost Savings**

By making the project freely available, this reduces licensing and acquisition costs.

- ☐ **Ease of Repurposing**

The open nature of the code allows users to modify and adapt the software to suit their specific needs, fostering customization and flexibility.

- ☐ **Minimization of Vendor Lock-in/Flexibility of Vendor Choice**

Users are not tied to a single vendor, providing the freedom to choose between different vendors.

- ☐ **Enable Transparency**

The source code is accessible and visible to anyone, promoting transparency in how the software functions, which helps build trust.

- ☐ **Enable extension/extensibility**

Users can extend and enhance the functionality of the software by adding their own features.

- ☐ **Increase Interoperability**

Planning in the open enables future compatibility and interoperability between different systems and software applications.

- ☐ **Facilitate Experimentation/Early Adoption**

Working in the open encourages experimentation and early adoption of cutting-edge technologies, leading to faster innovation and improvement in software capabilities.

## State the Risks of Open Sourcing the Project

- ☐ **Security Risks**

Vulnerabilities may be exposed if the code is not thoroughly reviewed, potentially leading to security breaches or exploitation. (See: [SECURITY.md](#)) Does this project connect to any CMS-internal only systems? Does this project require authorization or authentication to operate? Does this project detail any non-public directories of CMS/HHS systems or people?

- ☐ **Financial Risks**

Costs may arise from maintaining code, community engagement, addressing security concerns, subscription costs, hardware costs, specialized tooling or infrastructure costs among others. Does this project require any ongoing financial costs or subscription fees? (e.g., Cloud Hosting, Specialized build systems, paid maintainers, paid libraries or dependencies.)

- ☐ **Privacy Risks**

Does this project require access to non-public, non-synthetic PII, PHI, or other internal-only CMS systems containing such data or information?

## Questions

- Does the code contain or touch any private information such as Personal Identifiable Information (PII) or Protected Health Information (PHI)?
  - Can it be removed? Is it absolutely needed to function? Can it be shipped with synthetic data instead?
- Does the code interface with any of CMS' internal-only systems (e.g. mainframes, JIRA instances, databases, etc...)?
- Does the repository contain any keys or credentials to access or authenticate with CMS' systems?
  - Can it be removed or is it needed?

If you answered “yes” to any of the above questions, your project may be ‘sensitive’ in nature, and require a more thorough review before sharing publicly. Please reach out to [opensource@cms.hhs.gov](mailto:opensource@cms.hhs.gov) for guidance. If you answer yes to any of these questions above, it is best to seek guidance *before* releasing open source.

## Results

*Insert Review Here*

## Code Review

The existing codebase should be given a one time, top-to-bottom code quality and security vulnerability review by two (or more) engineers who have written production code within the past two years, in the languages used in the project. Engineers should review credential management practices with the development team to ensure that any keys, passwords, or other sensitive configurations are not checked into source code or in the git history.

The engineers can be federal government employees or trusted partners from outside the agency from other contracts, or from independent testing contracts. Their names, organizations, comments and approval/disapproval on the overall codebase should be tracked in this document.

To provide independent review, ideally the engineers should not have been involved in the development of the software product. This includes engineers who wrote part of the software or who directly provided technical direction and oversight in the creation of the software.

As part of the code review, engineers should reference modern listings of the most significant software security vulnerabilities. For instance, an acceptable description would be that the engineers showed how they used automated tools and manual review to check each item in [OWASP's current 10 Most Critical Web Application Security Risks](#).

## Results

*Insert Review Here*

## Code Analysis

A best practice and a requirement for releasing a repository open source at many parts of CMS is to run at least one automated tool for code analysis (such as static code analysis, repolinters, secret scanners) on the codebase to detect for security vulnerabilities or sensitive information, and results have been appropriately addressed. Even if all findings are eventually fixed, if the initial scans revealed significant, severe vulnerabilities (such as SQL injection vulnerabilities), this may indicate that the software development team was not adhering to the best practices required for open source public release, and should be given additional review.

Ideally, automated tooling for code analysis should be incorporated as a regularly scheduled part of the software development lifecycle. The development team should briefly document how frequently they commit to running these automated scanning tools, and how they will be running these tests, and who will be monitoring, interpreting, and acting upon the results.

## Toolkit

Below is a list of suggested tools to run for code analysis:

| Tool            | Description                                                                            | Link                                                                                                                                                                                                                                                            |
|-----------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repo Linter     | Lint repositories for common issues such as missing files etc.                         | <a href="https://github.com/todogroup/repolinter">https://github.com/todogroup/repolinter</a>                                                                                                                                                                   |
| gitleaks        | Protect and discover secrets using Gitleaks                                            | <a href="https://github.com/gitleaks/gitleaks">https://github.com/gitleaks/gitleaks</a><br>Use and run <a href="#">gitleaks.yml</a> provided in repository                                                                                                      |
| git filter-repo | Entirely remove unwanted files / files with sensitive data from a repository's history | <a href="https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository">https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository</a> |

## Results

*Insert Review Here*

## Review Licensing

Ensure that acceptable licensing is decided for the project. Most often, software released as open source by the federal government does so under the Creative Commons Zero 1.0 license.

Suggested licensing:

### Public Domain

This project is in the public domain within the United States, and copyright and related rights in the work worldwide are waived through the CC0 1.0 Universal public domain dedication.

All contributions to this project will be released under the CC0 dedication. By submitting a pull request, you are agreeing to comply with this waiver of copyright interest.

If your project is not being dedicated to the public domain under CC0, due to being work for hire, or some other documented reason, then choosing another [OSI approved license](#) is the next best thing.

## Results

*Insert Review Here*

## Review Commit History

Review the history of commits to the version control system used, and whether the team prefers to clean (e.g.,rebase) this history before releasing to the public.

If not rebasing, verify that:

1. There are no obscene or impolite remarks in comments or commit history
2. There are no sensitive internal URLs/IP addresses in comments or commit history
3. There are no credential files such as passwords, API/SSH/GPG keys checked into the repo

Consider using the following tools to perform the tasks above:

| Tool            | Description                                                                                             | Link                                                                                                                                                                                                                                                                                                                                            |
|-----------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gitleaks        | Open source tool that detects and prevents secrets (passwords/api/ssh keys) checked-in to your git repo | <a href="https://github.com/gitleaks/gitleaks">https://github.com/gitleaks/gitleaks</a><br><a href="https://akashchandwani.medium.com/what-is-gitleaks-and-how-to-use-it-a05f2fb5b034">https://akashchandwani.medium.com/what-is-gitleaks-and-how-to-use-it-a05f2fb5b034</a><br>Use and run <a href="#">gitleaks.yml</a> provided in repository |
| git filter-repo | Entirely remove unwanted files / files with sensitive data from a repository's history                  | <a href="https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository">https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository</a>                                                                                 |

## Results

*Insert Review Here*

## Review Repository Hygiene

As part of our repository hygiene requirements, the project must include certain files and sections. Using repolinter will help identify missing files and content that will need to be added to your repository before outboundng.

### Running repolinter on your repository locally

1. Install repolinter

```
npm install -g repolinter
```

2. Add [repolinter.json](#) to the root directory of your project

3. Run command:

```
repolinter lint .
```

4. The result produces a list of files section existence checks, indicating whether each requirement was met or not.

```
✓ license-file-exists: Found file (LICENSE.md)
✓ security-file-exists: Found file (SECURITY.md)
✓ readme-file-exists: Found file (README.md)
✓ contributing-file-exists: Found file (CONTRIBUTING.md)
✓ maintainers-file-exists: Found file (MAINTAINERS.md)
✖ codeowners-file-exists: Did not find a file matching the specified patterns ({docs/,.github/,}CODEOWNERS.md)
△ governance-file-exists: Did not find a file matching the specified patterns ({docs/,.github/,}GOVERNANCE.md)
✖ community-guidelines-file-exists: Did not find a file matching the specified patterns ({docs/,.github/,}COMM
✖ code-of-conduct-file-exists: Did not find a file matching the specified patterns ({docs/,.github/,}CODE_OF_C
```

Running repolinter on your repository via GitHub Actions

- 1. Add the tier-specific [repoHygieneCheck.yml](#) to the github directory of your project. The file includes a job that runs repolinter called [repolinter-checks](#)
- 2. Manually trigger the workflow
- 3. The results produces an issue on the repository with a list of file and section existence checks, indicating whether each requirement was met or not

Review Content

The project should include the following files and sections ([link to templates](#)):

- **[ ] LICENSE**  
*License of your project, whether public domain (CC0) or other OSI-approved License. Using 'vanilla' license text will allow for GitHub to auto-label the license information on the repository landing page.*
- **[ ] code.json**  
*Contains metadata about the project, refer to [Review Project Metadata](#)*
- **[ ] README.md**  
*An essential guide that gives viewers a detailed description of your project*

| Section             | Description                                                                                                                                                                                                                                                            | Included? |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Project Description | This should be 1-3 sentence short description of the project that can be used as a 'one-liner' to describe the repo. A best practice is using this same language as the official 'description' on a GitHub repo landing page.                                          | ✓ ✖       |
| About the Project   | This should be a longer-form description of the project. It can include history, background, details, problem statements, links to design documents or other supporting materials, or any other information/context that a user or contributor might be interested in. |           |
| Core Team           | This information helps with succession planning and provenance for security compliance and remediation. It helps future users and contributors understand where the code originated.                                                                                   |           |

| Section       | Description                                                                                                                                                                                                                                                                                     | Included? |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Policies      | This section is to explicitly link to Federal policies and guidelines that are required or recommended for Federal projects to comply with, such as Accessibility (508), Interoperability, Anti-deficiency, Security, Licensing, and other policies that can vary between agencies and domains. |           |
| Public Domain | A best practice is to list the LICENSE under which a project is released at the bottom of the README. In most cases for Federal repos, we default to Creative Commons Zero 1.0 International (world-wide public domain).                                                                        |           |

- **[ ] COMMUNITY.md**

*Outlines the community members, roles, responsibilities, and guidelines for participating in the project.*

| Section                  | Description                                                                                                                                                                                                                | Included |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Table of Project Members | Who are the points of contact in your project who are responsible/accountable for the project? This can often be an engineering or design manager or leader, who may or may not be the primary maintainers of the project. | ✓ ✗      |

- **[ ] CONTRIBUTING.md**

*Provides guidance on how users can run your project and make contributions to it*

| Section               | Description                                                                                                                                                                                                                                                                                          | Included |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Getting Started       | Includes installation steps, prerequisites for installation, and instructions for working with the source code.                                                                                                                                                                                      | ✓ ✗      |
| Building dependencies | This step is often skipped, so don't forget to include the steps needed to install on your platform. If your project can be multi-platform, this is an excellent place for first time contributors to send patches.                                                                                  |          |
| Building the Project  | Be sure to include build scripts and instructions, not just the source code itself!                                                                                                                                                                                                                  |          |
| Writing Issues        | Make a brief statement about where to file issues, and conventions for doing so. Link to ISSUE_TEMPLATE.md file.                                                                                                                                                                                     |          |
| Policies              | This section is here to explicitly link to Federal policies and guidelines that are required or recommended for Federal projects to comply with, such as Accessibility (508), Interoperability, Anti-deficiency, Security, Licensing, and other policies that can vary between agencies and domains. |          |
| Public Domain         | This section is to explicitly link to Federal policies and guidelines that are required or recommended for Federal projects to comply with, such as Accessibility (508), Interoperability, Anti-deficiency, Security, Licensing, and other policies that can vary between agencies and domains.      |          |

- **[ ] SECURITY.md**

*Details security policies and procedures*

- **[ ] repolinter.json**

*Lints repository for missing files and sections above*

## Communications Style Guide

As you are writing content about the repository, consider the following when describing the project, the agency, and how work is done:

- **Focus on solutions, not problems** *Rather than describing the negatives and shortcomings of an existing project, process, or group, focus on highlighting the benefits, outcomes, and positive impacts of the solution. Be constructive and accentuate the positive.*
- **Use Plain Language** *Be clear and concise when explaining concepts. Avoid jargon unless it is necessary and well-defined. Use active voice to be clearer and more engaging.*
- **Focus on the audience** *Write for a broad audience, ensuring that developers, stakeholders, end-users, and viewers of all skill levels can understand and engage with the content.*

Please refer to the style guides below for additional tips and guidance:

- CMS Resources
  - [Guidelines for Effective Writing](#)
  - [Tone of Voice section in Branding Guide](#)
  - [2020 Plain Writing Report](#)
- [18F Style Guide](#)
- [UK Content Style Guide](#)

## Results

*Insert Review Here*

## Review Project Metadata

As part of the [SHARE IT Act](#), [Federal Source Code Policy](#), and the agency's software inventory tracking initiatives, each repository must contain a [code.json file](#), storing metadata on your project.

For more information on code.json, please review the [gov-codejson documentation repository](#).

## Creating code.json on your repository

### Using form site

Users can fill out a web form that creates a code.json file to be uploaded to a project's source code repository: <https://dsacms.github.io/codejson-generator>.

### Using automated-codejson-generator

The [automated-codejson-generator](#) is a GitHub Action that automatically generates and maintains code.json files for federal open source repositories. It ensures schema consistency and automates various metadata calculations.



## Using the repo-scaffolder cookiecutter CLI

1. Install cookiecutter

```
python3 -m pip install --user cookiecutter
```

2. In the `.github` directory, run the command:

```
cookiecutter . --directory=codejson
```

3. Answer various questions about your project.
4. A code.json file will be generated with your responses.

As you continue development in this repository, it is important to keep this file up-to-date. Our [automated-codejson-generator](#) can assist with updating this file.

### Results

*Insert Review Here*

## Review Repository Details

The GitHub repository homepage features a concise description of the project, a list of relevant topic tags, and general information about the repository to provide a comprehensive overview for users and contributors.

### About Section:

- ☐ Description

*1-2 sentences describing the project*

- ☐ Website

*Link to project's website*

- ☐ Topics

*Tags for project discoverability. Helpful topics to classify a repository include the repository's intended purpose, subject area, community, or language.*

### Include in Home Page:

- ☐ Releases
- ☐ Packages
- ☐ Deployments

### Results

*Insert Review Here*

# Additional Notes & Questions

Insert any notes or questions here

## Sign off on risk acceptance of open-sourcing the software product

- **Date:** Add date
- **Repository Name:** Add repo-name

Before outbounding this repository, it's important that the appropriate stakeholders review and acknowledge the risks and responsibilities associated with releasing the code to the public. This step ensures transparency and accountability while enabling informed decision making.

### Security and Privacy Verification

- ☐ I acknowledge that this project does **NOT**:
  - ☐ contain any PII/PHI, or create an identifiable risk to the privacy of an individual.
  - ☐ interface with any CMS Internal Systems.
  - ☐ contain any keys or credentials to authenticate with CMS systems.

### National Security and Intelligence Verification

- ☐ I acknowledge that this project is **NOT**:
  - ☐ primarily for use in national security systems, as defined in Section 11103 of title 40, USC.
  - ☐ created by an agency or part of an agency that is an element of the intelligence community, as defined in section 3(4) of the National Security Act of 1947.
  - ☐ exempt under section 552(b) of title 5, USC (commonly known as the "Freedom of Information Act").

### Export and Regulatory Compliance

- ☐ I acknowledge that this project is **NOT** prohibited under:
  - ☐ Export Administration Regulations.
  - ☐ International Traffic in Arms Regulations (ITAR).
  - ☐ Regulations of the Transportation Security Administration related to the protection of sensitive information.
  - ☐ Federal laws and regulations governing the sharing of classified information.

If all boxes have been checked, please proceed to the **Flipping the Switch** section below this one, otherwise, this section must be filled out and approved by the indicated stakeholders before public release.

After reviewing the materials prepared by the team that is working to open source the product, the business owner signs off on a risk acceptance for open-sourcing the software product.

Requesting sign off from key people on this request.

| Reviewer Organization | Reviewer Name | Reviewer's Recommendation |
|-----------------------|---------------|---------------------------|
|-----------------------|---------------|---------------------------|

| Reviewer Organization          | Reviewer Name                                   | Reviewer's Recommendation                                                           |
|--------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------------|
| Code Reviewer's Recommendation | CODE REVIEWER 1 CODE REVIEWER 2 CODE REVIEWER 3 | [Approved/Needs Approval]<br>[Approved/Needs Approval]<br>[Approved/Needs Approval] |
| ISSO                           | ISSO REVIEWER                                   | [Approved/Needs Approval]                                                           |
| ISG Technical Approval         | ISG REVIEWER                                    | [Approved/Needs Approval]                                                           |
| Business Owner(s)              | BUSINESS OWNER 1<br>BUSINESS OWNER 2            | [Approved/Needs Approval]<br>[Approved/Needs Approval]                              |

## Flipping the Switch: Making the Repository Public

Once the repository has passed outbound review, we are ready “flip the switch” and officially make it public. Once made public, there are a couple of actions that need to be taken:

### Repository Actions

Please enable the following features to enhance repository security and maintain code quality:

- **[ ] Dependabot Alerts**

*A GitHub Feature. Get notified when one of your dependencies has a vulnerability*

- **[ ] Secret Scanning Alerts**

*A GitHub Feature. Get notified when a secret is pushed to this repository. Ideally set this up to run after each new commit is pushed to the Repository.*

- **[ ] Branch Protections**

*Ensures the integrity of important branches by preventing unauthorized actions like force pushes and requiring pull request reviews with specific checks before merging. Dev and main should be protected branches in the repository.*

- **[ ] Git Branching**

*After making the repository public, make sure there is a coherent git branching plan in place. For example: agree to merge feature related pull requests into dev but merge bug fixes into main instead of dev first.*

- **[ ] Add Repolinter GH Action to CI**

*For ongoing adherence to repository hygiene standards, integrate the [repolinter GitHub Action](#) into your CI pipeline. This addition enhances your workflow by automatically enforcing repository cleanliness standards.*

- **[ ] Optional: DCO (Developer Certificate of Origin)**

*Requires all commit messages to contain the Signed-off-by line with an email address that matches the commit author. The Developer Certificate of Origin (DCO) is a lightweight way for contributors to certify that they wrote or otherwise have the right to submit the code they are contributing to the project. The GitHub app to enforce DCO can be found [here](#) .*

## Communications & Rollout

Share the good news with communities both inside and outside CMS!

- **[ ] Draft a launch announcement**

Be sure to include the following information:

- Repo Description
  - Repo URL
  - Authoring Team Email Contact
  - Authoring Team URL
  - Authoring Team Slack Channel
  - Call to action (File issues, contribute PRs)
- **[ ] Post launch announcement to CMS slack channel**
  - #cms-opensource
  - #cms-api-community
  - #cms-data-community
  - #cms-engineering-community
  - #ai-community
- **[ ] Send a launch announcement email**
- **[ ] Add launch announcement to a Confluence Wiki Page**

## Tracking

Add your project to our inventories via code.json.

- **[ ] Add a completed code.json file to the repository**